Week 2 - Day 2 (Tuesday): Project Structure, Stubs and Configuration

Intern: Likhitha N | **Role:** IoT / Embedded | **Company:** CRASH CONNECT PVT LTD

> Hardware is not available, so hardware access is a **stub** or uses **simulated / recorded data**. Everything not implemented or not tested is labelled in the code output and in this document.

1. Project

ESP32-S3 project using the Week 1 setup (Arduino C++, 115200 baud Serial Monitor, TinyGPSPlus library). The Monday module list became the folder structure below. One folder per module, each with a header (`.h`) that defines its interface and a source file (`.cpp`) with the stub or implementation.

```
firmware/
  crashconnect_fw/
    crashconnect_fw.ino      main sketch: initialisation and main
loop
    MODULES.md               short notes on every module
    libraries.txt            TinyGPSPlus
    src/
      common/cc_types.h      shared enums: device states, event
types
      config/                configuration structure and defaults
      gnss/                  NMEA parsing, fix validity
      imu/                   motion samples, acceleration magnitude
      comms/                 A7670C communication (stub)
      safety/                impact / SOS evaluation (placeholder
rule)
      sos/                   SOS button
      power/                 battery voltage (stub)
      fault/                 fault flags
      status/                device status summary
      state/                 device state machine
      telemetry/             telemetry packet builder
      sim/                   SIMULATED test inputs (test only)
  host_tests/                PC test programs for the same code
  platformio.ini             optional PlatformIO project file (not
tested here)
```

2. Module responsibilities and status

Module	Responsibility	Status
config	One place for device ID, communication settings, intervals, timeouts, thresholds and pins	Implemented (values are drafts)
gnss	Feed NMEA text in, checksum check, decode GGA/RMC (GP and GN talkers), report a fix only if valid and recent	Parsing implemented and tested with recorded data. UART read from the MAX-M10S is a **stub**
imu	Provide samples and acceleration magnitude	Simulated test values implemented. LSM6DSOX register read is a **stub** (I2C or SPI Pending Confirmation)
comms	Connect and send packets	**Stub**: pretends to send and logs it. A7670C AT commands, MQTT/API Pending Confirmation
safety_event	Impact or SOS event decision	**Placeholder** rule: SOS, or acceleration magnitude at or above a threshold. Not real crash detection

sos	SOS button	Simulated input implemented. Real button read is a **stub** (pin, debounce, long press Pending Confirmation)
power	Battery voltage	**Stub**: always unavailable (ADC pin and divider Pending Confirmation)
fault	Fault flags, critical or not	Implemented
status	One-line status summary	Implemented
state	Device state machine	Implemented on Day 3 (see Day 3 document)
telemetry	Packet builder	Started on Day 3 (see Day 3 document)
sim	Test inputs for the demonstration	Implemented, test only

3. Configuration structure

All values live in `src/config/config.cpp` and are read through `config()`. The logic code does not contain hard-coded numbers.

Group	Items	Notes
Device	device ID, firmware version, packet version, simulated-inputs switch	Device ID is a draft value
Communication	transport, APN, server host, port, telemetry topic, event topic	All **PENDING_CONFIRMATION**
IMU interface	pending / I2C / SPI	Set to pending
Timing	main tick, boot and self-test time, IMU sample interval, GNSS poll interval, telemetry interval, event telemetry interval, GPS and network timeouts, GNSS stale time, event hold time, fault restart delay	Draft values
Threshold	impact acceleration (g)	Placeholder value, not a decided crash threshold
Pins	GNSS RX/TX, IMU SDA/SCL/CS, modem RX/TX/PWRKEY, SOS button, battery ADC	All -1 = not assigned (Pending Confirmation)

`config_is_valid()` checks that the configuration makes sense (device ID present, intervals greater than zero). It is used by the self-test.

4. How to build and run

- **Arduino IDE:** install the library *TinyGPSPlus* (Mikal Hart), open `crashconnect_fw/crashconnect_fw.ino`, choose an ESP32-S3 board and open the Serial Monitor at 115200 baud. The `src/` sub-folders are compiled automatically.
- **PC test (what I ran):** `firmware/host_tests/run_tests.sh` builds the same source files with g++ against the real TinyGPS++ source and a tiny Arduino stand-in, runs 66 unit tests and the full demonstration scenario.

I could **not** compile with the ESP32-S3 toolchain or run Wokwi in my workspace. The ESP32 build and the Wokwi screenshots still have to be done on the intern laptop. Whether Wokwi accepts the `src/` sub-folders as they are has not been tested; if it does not, use the Arduino IDE or PlatformIO.

5. Pending Confirmation (Day 2)

- Pin assignments, GNSS interface (UART or I2C), IMU interface (I2C or SPI).
- A7670C start-up sequence, APN, MQTT or API.
- Whether network connection should start in parallel with GPS acquisition (right now it starts after the GPS fix).

The full list is in `Pending_Confirmation.pdf`.